

Plantilla multi-formato para investigación aplicada en buenas prácticas de desarrollo de software

Samuel Felipe Paloma Quila

SENA

Análisis y Desarrollo de Software

Ciudad, Colombia

samupalo3@gmail.com

Resumen

El presente artículo científico presenta en detalle el diseño, la implementación y la validación de un sistema de gestión de tickets, ofreciendo una solución tecnológica para optimizar el flujo de trabajo en instituciones donde la atención a incidencias y requerimientos se maneja de forma manual o poco estructurada. El proyecto propone una arquitectura híbrida que integra un portal web y una aplicación móvil Android, ambos conectados a una base de datos centralizada, con módulos de seguridad, notificaciones en tiempo real y funcionalidades para la clasificación automática de tickets. El documento describe las etapas de desarrollo y se fundamenta en ocho artículos seleccionados que abordan frameworks de desarrollo modernos, metodologías ágiles y patrones arquitectónicos relevantes. Estos estudios incluyen aportes sobre frameworks PHP como Laravel y Symfony [1], [7], bibliotecas front-end como React y Vue [2], [4], [5], React Native para aplicaciones móviles [3], y enfoques metodológicos basados en Scrum y en el conjunto de metodologías ágiles [6], [9]. A partir de este contraste, se evidencian las ventajas de integrar teoría y práctica en un mismo proyecto, validando que la incorporación de frameworks modernos y enfoques ágiles permite obtener aplicaciones escalables, seguras y con una alta orientación hacia la experiencia de usuario.

Keywords

buenas prácticas, ingeniería de software, investigación aplicada, reproducibilidad

1. Introducción

El manejo de incidentes supone un reto importante para las instituciones, sean públicas o privadas. En muchos casos, los trámites se llevan a cabo manualmente, mediante correos electrónicos dispersos, llamadas telefónicas o mediante whatsapp. Esto conduce a una falta de trazabilidad, pérdida de información y retrasos importantes en la atención a las solicitudes. Esta situación, expuesta en el Informe de Especificación de Requerimientos del Sistema de Gestión de Tickets [10], sirvió como punto de partida para la propuesta de un proyecto con el propósito de digitalizar y mejorar estos procedimientos.

El propósito del sistema propuesto es atender los siguientes requerimientos: centralización de datos, alertas en tiempo real, paneles con métricas confiables y la posibilidad de personalizar según los roles de los usuarios (técnicos, administradores, superadministradores y funcionarios). La arquitectura tecnológica establecida, que combina un sitio web basado en React con un backend construido en Laravel y una aplicación móvil para técnicos, permite

garantizar una comunicación ininterrumpida y un monitoreo riguroso de las solicitudes.

En el marco teórico, se consultaron artículos académicos que sirven como referencia para justificar las decisiones tecnológicas y metodológicas. Por ejemplo, el estudio sobre Laravel [1] recalca la relevancia.^o en el estudio sobre Laravel [1] se recalca la relevancia de este framework en términos de seguridad y rapidez, mientras que el trabajo comparativo entre Laravel y Symfony [7] permitió confirmar que, aunque Symfony ofrece mayor robustez empresarial, Laravel resulta más accesible y flexible para un equipo en formación. Del mismo modo, React se presenta como una biblioteca flexible y fácil de adoptar, lo que coincide con investigaciones que la destacan como la opción preferida frente a Angular y Vue en numerosos proyectos [2], [4], [5]. React Native, por su parte, constituye una herramienta estratégica para futuras ampliaciones hacia entornos multiplataforma [3].

El proyecto se nutrió también de los aportes de metodologías ágiles. El marco de trabajo Scrum se utilizó como base para organizar el flujo de trabajo, asegurando entregas incrementales y revisiones continuas [6]. Sin embargo, tal como lo advierte la literatura sobre metodologías ágiles [9], la adopción de estos enfoques debe complementarse con unas buenas prácticas de ingeniería de software para evitar problemas y asegurar la escalabilidad del sistema.

El objetivo de este artículo es no solo presentar los progresos logrados en el desarrollo del sistema de gestión de tickets, sino también llevar a cabo un análisis comparativo con la literatura disponible para confirmar las decisiones tomadas en términos tecnológicos y metodológicos y establecer áreas para futuras investigaciones.

2. Marco teórico y trabajos relacionados

Se consultaron artículos académicos que sirven como referencia para justificar las decisiones tecnológicas y metodológicas. El estudio sobre Laravel [1] recalca la relevancia de este framework en términos de seguridad y rapidez, mientras que el trabajo comparativo entre Laravel y Symfony [7] permitió confirmar que, aunque Symfony ofrece mayor robustez empresarial, Laravel resulta más accesible y flexible para un equipo en formación.

Del mismo modo, React se presenta como una biblioteca flexible y fácil de adoptar, lo que coincide con investigaciones que la destacan como la opción preferida frente a Angular y Vue en numerosos proyectos [2], [4], [5]. React Native, por su parte, constituye una herramienta estratégica para futuras ampliaciones hacia entornos multiplataforma [3].

Respecto a metodologías ágiles, el marco de trabajo Scrum se utilizó como base para organizar el flujo de trabajo, asegurando entregas incrementales y revisiones continuas [6]. Como lo advierte la literatura sobre metodologías ágiles [9], su adopción debe complementarse con buenas prácticas de ingeniería para asegurar escalabilidad.

3. Metodología

La metodología implementada para la realización del sistema, se fundamentó con el marco de trabajo Scrum, reconocido por su flexibilidad y capacidad de adaptación [6]. Se organizaron ciclos de trabajo denominados sprints de dos semanas, en los cuales el equipo debía entregar incrementos funcionales del sistema. Se realizaron reuniones de planificación, reuniones diarias (daily scrum) para el seguimiento del progreso, revisiones de sprint y retroalimentación, con el fin de garantizar una mejora continua. Este enfoque se complementa con documentación formal y un modelo de especificación de requerimientos detallado, tal como se establece en el Informe de Requerimientos [10].

Desde el punto de vista técnico, la elección de las herramientas y los frameworks se realizó con base en análisis y comparaciones realizadas. Laravel fue adoptado como framework backend debido a su simplicidad, seguridad integrada y ecosistema de herramientas como Eloquent ORM, Blade y migraciones [1]. Aunque Symfony se consideró por su robustez y orientación empresarial [7], se determinó que Laravel respondía mejor a los objetivos del proyecto. En el frontend, React fue implementado por su capacidad de reutilización de componentes, eficiencia y soporte por parte de una amplia comunidad [2]. Estudios comparativos demuestran que React ofrece un balance adecuado entre curva de aprendizaje y flexibilidad, en contraste con Angular, que presenta mayor complejidad [4], [5].

La aplicación móvil se diseñó únicamente para el sistema operativo Android, tomando en cuenta la accesibilidad de esta plataforma en el contexto del proyecto. Se contempló la futura incorporación de React Native para extender la compatibilidad hacia iOS y aprovechar las ventajas de compartir código entre plataformas [3].

El sistema está constituido por tres módulos fundamentales: la base de datos centralizada, la aplicación móvil y el portal web. El portal web facilita la administración, el registro de tickets y la creación de informes; la aplicación móvil brinda a los técnicos la posibilidad de recibir notificaciones push y guardar evidencias multimedia; y contar con una base de datos centralizada garantiza que la información sea íntegra, se pueda rastrear y esté disponible. La integración de estos módulos se llevó a cabo de acuerdo con el modelo arquitectónico MVC [8], lo cual permitió dividir las responsabilidades y hacer que el sistema pueda escalar a un futuro.

4. Implementación

El sistema está constituido por tres módulos fundamentales: la base de datos centralizada, la aplicación móvil y el portal web. El portal web facilita la administración, el registro de tickets y la creación de informes; la aplicación móvil brinda a los técnicos la posibilidad de recibir notificaciones push y guardar evidencias multimedia; y contar con una base de datos centralizada garantiza que la información sea íntegra, se pueda rastrear y esté disponible. La

integración de estos módulos se llevó a cabo de acuerdo con el modelo arquitectónico MVC [8], lo cual permitió dividir las responsabilidades y hacer que el sistema pueda escalar a un futuro.

5. Resultados

Los resultados obtenidos tras la implementación del sistema reflejan avances significativos respecto a la situación inicial descrita en el Informe de Requerimientos [10]. Entre los logros más relevantes se destacan:

1. Centralización del registro de incidencias, lo que evita duplicidad de solicitudes y mejora la trazabilidad de cada caso.
2. Implementación de un sistema de roles que diferencia entre funcionarios, técnicos, administradores y superadministradores, garantizando un control personalizado del sistema.
3. Incorporación de notificaciones en tiempo real, que facilita la comunicación inmediata entre usuarios, técnicos y administradores.
4. Inclusión de un panel de métricas para administradores, que permite medir tiempos de atención, evaluar desempeño y generar reportes.
5. Desarrollo de un módulo de clasificación automática de tickets, optimizando la asignación de recursos.

En términos de comparación con la literatura, estos resultados demuestran que Laravel cumple con las expectativas en materia de seguridad y rapidez [1], mientras que React garantiza una experiencia de usuario fluida y altamente interactiva [2]. Asimismo, la aplicación de Scrum aportó la flexibilidad necesaria para adaptarse a cambios en los

requerimientos, lo cual coincide con lo planteado en estudios recientes sobre metodologías ágiles [9].

6. Discusión

El análisis comparativo entre los resultados del proyecto y los hallazgos de la literatura evidencia coincidencias y aprendizajes relevantes. En primer lugar, la selección de Laravel como framework backend se justifica no solo por su facilidad de uso, sino también porque los estudios demuestran su capacidad para garantizar sistemas robustos y escalables en entornos de producción [1]. En comparación con Symfony, Laravel presenta una curva de aprendizaje más corta, lo que lo convierte en la opción ideal para equipos pequeños o en formación [7].

En segundo lugar, el uso de React permitió construir una interfaz más moderna y basada en componentes reutilizables, lo que coincide con estudios que resaltan su liderazgo en proyectos de gran escala [2], [4], [5]. La incorporación de React Native se vislumbra como una evolución natural del proyecto, en línea con experiencias previas que destacan su eficacia en el desarrollo multiplataforma [3].

Respecto a las metodologías ágiles, el marco de trabajo Scrum resultó útil para mantener el ritmo del proyecto; no obstante, como lo señalan autores expertos, su uso debe ir acompañado de prácticas robustas de documentación, arquitectura y pruebas constantes [6], [9]. Esta combinación fue fundamental para disminuir la deuda técnica y garantizar que el software tenga calidad.

Finalmente, en el proyecto se confirma que la integración de una teoría y práctica, se fortalece en el desarrollo de soluciones tecnológicas. Los artículos revisados no solo ayudaron como referencias bibliográficas, sino que guiaron las decisiones estratégicas durante el proceso de implementación, lo cual se tradujo en un producto funcional, seguro y alineado con las tendencias actuales del desarrollo de software.

7. Conclusiones

El desarrollo del sistema de gestión de tickets permitió comprobar que la integración de frameworks modernos como Laravel y React, junto con la aplicación de metodologías ágiles como Scrum, es una estrategia efectiva para resolver problemáticas de trazabilidad y eficiencia en el manejo de incidencias. La comparación con la literatura académica revisada confirma que estas herramientas y enfoques garantizan aplicaciones escalables, seguras y centradas en la experiencia del usuario.

Entre los principales aportes del proyecto se destacan: la centralización de información, la reducción en tiempos de atención, la personalización de roles y la posibilidad de generar métricas confiables para la toma de decisiones. Asimismo, el sistema constituye

una base sólida sobre la cual es posible implementar mejoras futuras o implementar nuevas funciones, como la ampliación hacia plataformas iOS mediante React Native, la creación de módulos de inteligencia artificial y la exploración de arquitecturas basadas en microservicios.

Este artículo evidencia que el uso de frameworks modernos no es una moda pasajera, sino una necesidad en el desarrollo de software competitivo y orientado a la calidad. Igualmente, se demuestra que las metodologías ágiles, aplicadas con criterio y disciplina, potencian la adaptabilidad, funcionamiento y aseguran productos más acordes con las necesidades reales de los usuarios finales.

A. Checklist de reproducibilidad (plantilla)

- **Datos:** fuente, versión, licencias, anonimización.
- **Código:** repositorio, commit hash, instrucciones de ejecución.
- **Entorno:** SO, versión de compiladores, dependencias, semillas.
- **Procedimiento:** pasos exactos para replicar resultados.
- **Resultados:** tablas/figuras generadas automáticamente en build/.

Referencias